

Python

Summary Session 15



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос



Что такое CSV?



CSV (Comma-Separated Values)

Это простой текстовый формат для хранения табличных данных. Каждая строка представляет собой одну запись, а значения разделяются запятыми, или другими разделителями

Пример CSV-файла

```
name,age,city  
Alice,30,New York  
Bob,25,London  
Charlie,35,Paris
```

Экспресс-опрос

?

Какие особенности CSV вы помните?

Особенности CSV



Каждая строка файла представляет собой запись



Поля внутри записи разделяются запятыми



Первая строка файла часто содержит заголовки колонок



Поддерживает как числовые, так и текстовые данные, но все трактуются как строковый тип

Использование CSV



Хранение и передача данных между системами



Импорт и экспорт данных в табличных редакторах (например, Excel)



Обмен данными между различными приложениями

Экспресс-опрос



Вспомните основные отличия между CSV и JSON

Сравнение CSV и JSON

Характеристика	CSV	JSON
Структура данных	Табличная структура с фиксированными колонками	Иерархическая структура с вложенными объектами и массивами
Типы данных	Все данные обычно строки, требуется преобразование	Поддержка различных типов данных (строки, числа, булевы значения, списки, объекты)
Читаемость	Легко читается человеком, особенно для простых данных	Легко читается подготовленным человеком, знакомым с иерархической структурой, но структура может быть громоздкой
Поддержка пропущенных значений	Пропущенные значения обычно представлены как пустые строки	Поддержка <code>null</code> для обозначения отсутствующих значений
Размер файла	Обычно меньше для простых данных	Может быть больше из-за метаданных и структуры

Сравнение CSV и JSON

Характеристика	CSV	JSON
Инструменты для работы	Модуль csv в Python, Excel	Библиотеки json в Python, JavaScript
Производительность при чтении/записи	Обычно быстрее для больших объемов данных	Может быть медленнее из-за необходимости разбора структуры
Использование	Хранение и обмен табличными данными	Хранение и обмен структурированными данными, конфигурации, API
Поддержка вложенных структур	Нет	Да
Стандартизация	Широко используемый стандарт	Широко используемый стандарт

Важно



Python поддерживает работу с CSV через встроенный модуль csv

Чтение CSV-файла

```
import csv
with open('sales.csv') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Экспресс-опрос



Что такое Reader?



Reader

Это специальный объект, который
построчно считывает данные из файла и
выдает их в виде списка строк

Разделитель



По умолчанию

По умолчанию в формате CSV используется запятая как разделитель между данными в строке

Указать другой

```
delimiter=';'
```

Запись в CSV-файл

```
with open('output.csv', 'w', newline='') as f:  
    writer = csv.writer(f)  
    writer.writerow(['name', 'age'])  
    writer.writerow(['Alice', 30])
```

Экспресс-опрос

?

Для чего используется атрибут `newline=""`?

Важно



Атрибут `newline=' '` стоит указывать, чтобы в файле не было лишних пустых строк

Метод writerows()



Запись списка строк

```
rows = [
    ['Alice', 30],
    ['Bob', 25],
    ['Charlie', 35]
]
with open('output.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['name', 'age']) #
заголовок
    writer.writerows(rows)           # сразу
все строки
```

Добавление строк в существующий файл

```
with open('sales.csv', 'a', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['Orange', 'Rainbow',
                    '2025-07-04', '110', '2.00'])
```

Экспресс-опрос

?

Чем станут заголовки столбцов при считывании данных из csv-файла в словарь?

Работа со словарями



Можно считывать данные из csv-файла в словарь. В этом случае заголовки столбцов станут ключами значений в каждой строке

Пример считывания в словарь

```
with open('sales.csv') as f:  
    reader = csv.DictReader(f)  
    for row in reader:  
        print(row)
```

Запись словаря в CSV-файл



Пример

```
with open('output.csv', 'w',  
        newline='') as f:  
    writer = csv.DictWriter(f,  
        fieldnames=['name', 'age'])  
    writer.writeheader()  
    writer.writerow({'age': 30, 'name':  
        'Alice'})
```

Пояснения

- Аргумент `fieldnames` обязателен: он задаёт порядок столбцов и их названия
- `writeheader()` необходим для записи строки с заголовком

Запись в файл списка словарей



Пример

```
new_persons = [
    {'name': 'David', 'age': 28},
    {'age': 34, 'name': 'Jane'},
    {'name': 'Anna', 'city': 'Berlin'},
    {'name': 'Martin', 'age': 20, 'city':
'London'}
]

with open('output.csv', 'a', newline='') as f:
    writer = csv.DictWriter(f,
        fieldnames=['name', 'age'],
        extrasaction='ignore')
    writer.writerows(new_persons)
```

Пояснения

- У разных словарей разные ключи и их порядок
- Атрибут 'a' при открытии файла – новые строки будут дописаны в конце этого файла
- writeheader() не вызываем, чтобы избежать повторной записи строки с заголовками
- Атрибут extrasaction='ignore' функции DictWriter() позволяет игнорировать лишние ключи в записываемых словарях, если их нет в списке fieldnames

Экспресс-опрос

?

Что такое пропущенные значения и как их можно обрабатывать?

Работа с пропущенными значениями



В CSV-файлах пропущенные обозначаются как два разделителя подряд (обычно – две запятых подряд) или разделителем, после которого следует конец строки

Пример обработки пропущенных значений

```
with open('output.csv') as f:
    reader = csv.reader(f)
    good_rows = []
    for row in reader:
        # Проверяем, есть ли в строке пропущенные значения
        if any(field == '' for field in row):
            print("Пропущенные значения обнаружены, строка пропущена:",
row)
            continue
        good_rows.append(row)

print("\nСтроки с полными данными:")
for row in good_rows:
    print(*row)
```

Экспресс-опрос

?

Какие недостатки модуля csv вы помните?

Недостатки модуля CSV

-  Отсутствие обработки пропущенных значений
-  Нет поддержки различных типов данных
-  Ограниченная обработка ошибок
-  Нет поддержки сложных структур данных
-  Ограниченные возможности фильтрации и агрегации данных

Важно



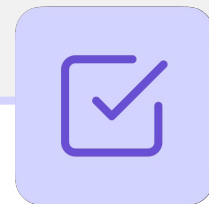
Для более продвинутой работы с CSV-файлами и данными в целом рекомендуется использовать специализированные библиотеки, например, модуль pandas.

Экспресс-опрос



Какой модуль используется для получения основных статистических характеристик числовых данных и какие его функции вы помните?

Важно



Модуль `statistics` входит в стандартную библиотеку Python и используется для получения основных статистических характеристик числовых данных

Среднее арифметическое – mean()

Основные статистические функции



Пояснение

Обычное среднее значение: сумма всех чисел, делённая на их количество

Формула

$$\text{mean} = \frac{x_1 + x_2 + \dots + x_n}{n}$$

Среднее геометрическое – `geometric_mean()`

Основные статистические функции



Пояснение

Уместно, когда усредняемые значения объединяются посредством умножения

Формула

$$\text{geometric mean} = \sqrt[n]{x_1 \cdot x_2 \cdot \dots \cdot x_n}$$

Среднее гармоническое – harmonic_mean()

Основные статистические функции



Пояснение

Используется, когда уместнее учитывать
обратные значения

Формула

$$\text{harmonic mean} = \frac{n}{\frac{1}{x_1} + \frac{1}{x_2} + \dots + \frac{1}{x_n}}$$

Мода – mode()

Основные статистические функции



Определение

Самое часто встречающееся значение в наборе данных

Пояснение

В случае нескольких мод — возвращает первую найденную. Для числовых и категориальных данных

Медиана – median()

Основные статистические функции



Определение

Значение, находящееся посередине
отсортированного ряда

Пояснение

Если чисел чётное количество —
берётся среднее из двух центральных.
Медиана устойчива к выбросам

Квантиль – quantiles()

Основные статистические функции



Определение

Делит упорядоченные данные на равные части

Пояснение

Например, квартиль — это деление на 4 части:

- 1-й квартиль (квантиль 25%) — значение, ниже которого лежит четверть данных
- 2-й квартиль — медиана
- 3-й квартиль (квантиль 75%) — значение, ниже которого лежит три четверти данных

Стандартное отклонение – stdev()

Основные статистические функции



Пояснение

Показывает, насколько значения в выборке отклоняются от среднего

Формула

$$\text{stdev} = \sqrt{\frac{1}{n-1} \sum (x_i - \bar{x})^2}$$

Дисперсия – variance()

Основные статистические функции



Пояснение

Квадрат стандартного отклонения.
Характеризует разброс данных относительно среднего

Формула

$$\text{variance} = \frac{1}{n-1} \sum (x_i - \bar{x})^2$$

Пример использования

```
import statistics

# Набор данных
data = [12, 15, 14, 10, 1, 18, 17, 15, 15, 14, 200]

# Средние значения
print("Арифметическое среднее:", statistics.mean(data))
print("Геометрическое среднее:", statistics.geometric_mean(data))
print("Гармоническое среднее:", statistics.harmonic_mean(data))

# Характеристики на отсортированном наборе
print("Отсортированный список:", sorted(data))
print("Медиана:", statistics.median(data))
print("Мода:", statistics.mode(data))

# Квантильные значения (делим на 4 части – квартиль)
print("Квартили:", statistics.quantiles(data, n=4))

# Измерения разброса
print("Стандартное отклонение:", statistics.stdev(data))
print("Дисперсия:", statistics.variance(data))
```

Экспресс-опрос

?

Что такое итератор и итерируемый объект?



Итерируемый объект

Это объект, который может состоять из множества элементов и предоставляет их по одному, когда это необходимо



Итератор

Это объект, который позволяет поочерёдно получать элементы коллекции без необходимости загружать их все в память сразу

Итератор содержит



Ссылку на итерируемый объект

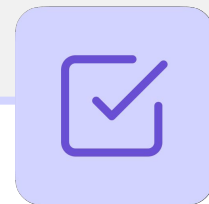


Текущую позицию



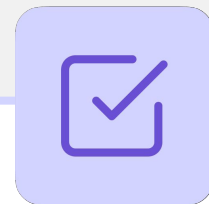
Логику получения следующего элемента

Важно



Итератор не изменяет коллекцию, на которую ссылается, а просто управляет процессом последовательного извлечения данных.

Важно



Итераторы используются в ситуациях, когда обработка данных должна выполняться эффективно и без лишнего расхода памяти.

Экспресс-опрос

?

Что такое магические методы?



Магические методы

Это специальные методы, которые вызываются автоматически при использовании встроенных механизмов языка, например цикла `for`. Другое название – dunder-методы (сокращение от `double underscore`). Их названия начинаются и заканчиваются двойным подчёркиванием

Магические методы для работы с итерацией

1

`__iter__()`

создаёт объект итератора из итерируемого объекта

2

`__next__()`

выдаёт следующий элемент итерируемого объекта

Пример: создание объекта итератор

```
numbers = [10, 20, 30] # Обычный список
iterator = numbers.__iter__() # Получаем итератор

print(iterator) # Итератор для списка
print(list(iterator)) # Преобразование в список

print(list(iterator)) # Второй раз список будет пустой
```

Особенности `.__iter__`



- `numbers.__iter__()` создаёт итератор, связанный со списком.
- Второй раз список будет пустой, так как итератор "потрачен"

Пример: последовательное получение элементов

```
numbers = [10, 20, 30]
iterator = numbers.__iter__() # Создаём итератор

print(iterator.__next__()) # Получаем нулевой элемент
print(iterator.__next__()) # Получаем первый элемент
print("-" * 30) # Можно прерваться на другие действия
print(iterator.__next__()) # Получаем второй элемент
```

Особенности `__next__`



- `iterator.__next__()` возвращает следующий элемент
- При каждом вызове итератор запоминает текущую позицию

Экспресс-опрос

?

Что делают встроенные функции, одноименные магическим методам?

Одноименные встроенные функции

1

`iter(obj)`

вызывает `obj.__iter__()`, создавая итератор из итерируемого объекта

2

`next(iterator)`

вызывает `iterator.__next__()`, возвращая следующий элемент

Пример использования встроенных функций

```
numbers = [1, 2, 3] # Итерируемый объект  
  
iterator = iter(numbers) # Вызывает numbers.__iter__()  
  
print(next(iterator)) # Вызывает iterator.__next__()  
print(next(iterator)) # Вызывает iterator.__next__()  
print(next(iterator)) # Вызывает iterator.__next__()
```

Экспресс-опрос

?

Какая функция позволяет измерять, сколько памяти занимает объект в байтах?



Функция `sys.getsizeof()`

Эта функция позволяет измерять, сколько памяти занимает объект в байтах. Это полезно для сравнения объектов

Пример: сравнение списка и итератора

```
import sys

# Список из 1 000 000 чисел
numbers_list = [x for x in range(1_000_000)]
print("Размер списка:", sys.getsizeof(numbers_list), "байт")

# Итератор, из списка
numbers_iterator = numbers_list.__iter__()
print("Размер итератора:", sys.getsizeof(numbers_iterator), "байт")
```

Экспресс-опрос

?

Когда возникает ошибка StopIteration?



Ошибка StopIteration

Такая ошибка возникает когда итератор исчерпал все доступные элементы, и метод `__next__()` не может вернуть следующий

Пример

```
numbers = [1, 2, 3]
iterator = iter(numbers) # Создаём итератор

print(next(iterator))
print(next(iterator))
print(next(iterator)) # Последний элемент
print(next(iterator)) # Ошибка StopIteration
```

Вывод

```
1
2
3
```

```
-----
StopIteration                                Traceback (most recent call last)
Cell In[52], line 7
      5 print(next(iterator))
      6 print(next(iterator)) # Последний элемент
----> 7 print(next(iterator)) # Ошибка StopIteration

StopIteration:
```

Экспресс-опрос

?

Как избежать ошибки StopIteration?

Как избежать появления ошибки StopIteration



Объяснение

Функция `next()` поддерживает второй аргумент, который возвращается вместо ошибки `StopIteration`, если элементы в итераторе закончились

Пример

```
numbers = [1, 2, 3]
iterator = iter(numbers)

print(next(iterator))
print(next(iterator))
print(next(iterator))
print(next(iterator, None)) # None вместо
StopIteration
print(next(iterator, 0)) # 0 вместо
StopIteration
```



Цикл for

В Python этот цикл используется для перебора итерируемых объектов (списки, строки, итераторы и т.д.)

Экспресс-опрос

?

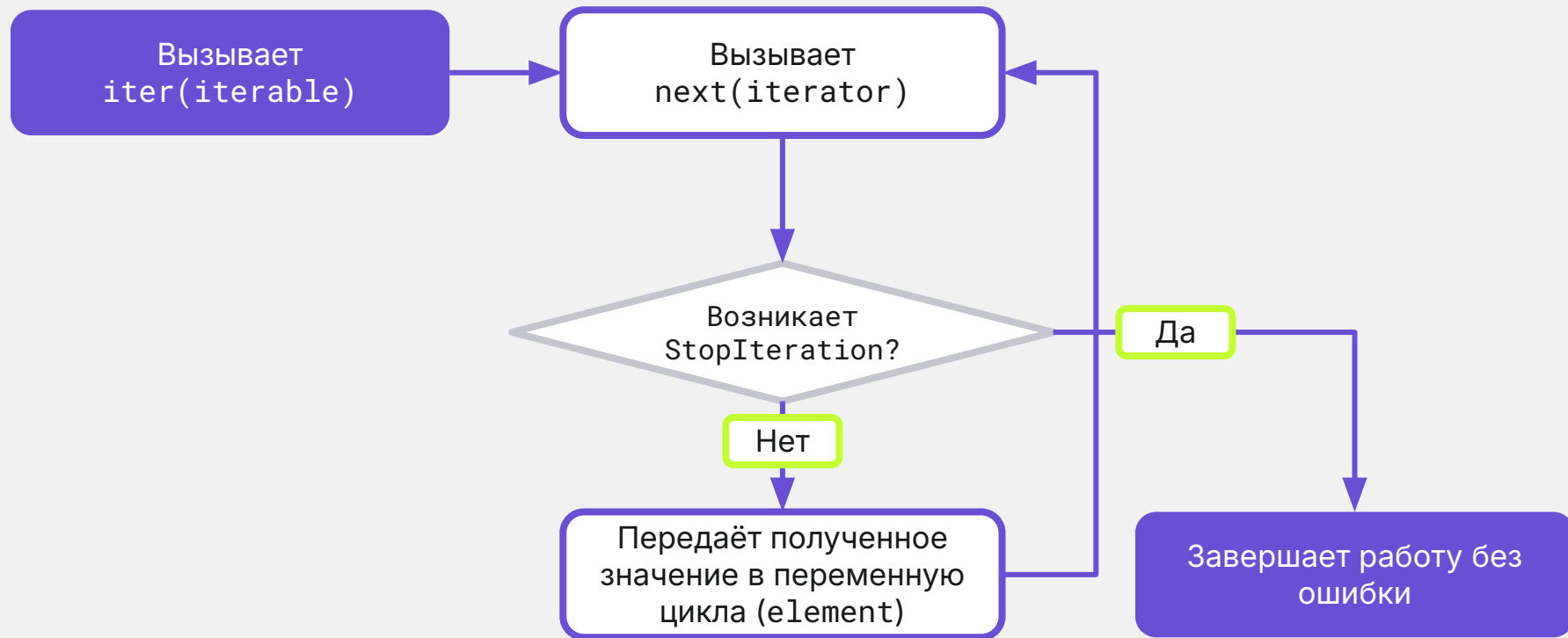
Что использует цикл for внутри себя?

Важно



Внутри себя `for` автоматически использует итератор, вызывая методы `__iter__()` и `__next__()`

Пошаговая работа цикла for



Пример работы цикла for

```
numbers = [10, 20, 30] # Итерируемый объект
```

```
# Цикл for по итерируемому объекту
```

```
for num in numbers:
    print(num)
```

```
print()
```

```
iterator = iter(numbers) # Итератор
```

```
# Цикл for по итератору
```

```
for num in iterator:
    print(num)
```

Как Python выполняет этот цикл внутри

```

numbers = [10, 20, 30] # Итерируемый объект
iterator = iter(numbers) # Создание итератора

while True:
    try:
        num = next(iterator) # Получаем следующий элемент
        print(num)
    except StopIteration:
        break # Завершаем цикл при окончании элементов
    
```

Экспресс-опрос

?

Какие особенности итераторов вы помните?

Особенности итераторов



Экономия памяти



Последовательный доступ



Неизменяемость



Однократное использование



Модуль itertools

Этот модуль предоставляет набор инструментов для работы с итераторами, позволяя создавать эффективные итерируемые последовательности, объединять, фильтровать и изменять потоки данных без лишнего расхода памяти

Экспресс-опрос

?

Какие функции модуля `itertools` вы знаете?

Бесконечная последовательность чисел: count()

Примеры использования itertools.

```
import itertools

counter = itertools.count(start=1, step=10)

print(next(counter))
print(next(counter))
print(next(counter))
print(next(counter))
print(next(counter))
```

Бесконечное повторение элементов: cycle()

Примеры использования itertools.

```
cycler = itertools.cycle(["A", "B", "C"])

print(next(cycler))
print(next(cycler))
print(next(cycler))
print(next(cycler)) # A (повторяется)
print(next(cycler)) # B (повторяется)
```

Объединение итерируемых объектов: chain()

Примеры использования itertools.

```
merged = itertools.chain([1, 2, 3], [100, 200, 300])  
  
print(list(merged))
```

Декартово произведение: product()

Примеры использования itertools.

```
pairs = itertools.product([1, 2, 3], ["A", "B", "C"])  
print(pairs)  
print(list(pairs))
```

Декартово произведение: product()

Примеры использования itertools.

```
pairs = itertools.product([1, 2, 3], ["A", "B", "C"])  
print(pairs)  
print(list(pairs))
```



Функция `itertools.permutations()`

Эта функция создаёт все возможные упорядоченные перестановки элементов. Можно указать длину перестановки (r), или использовать всю последовательность по умолчанию

Перестановки элементов: permutations()

Примеры использования itertools.

```
letters = ["A", "B", "C"]

# Все возможные перестановки
perms = itertools.permutations(letters)
print(list(perms))

# Все возможные перестановки длины 2
perms = itertools.permutations(letters, 2)
print(list(perms))
```

Экспресс-опрос



Что такое генератор?



Генератор

Это специальный вид итератора, который создаёт элементы по запросу, вместо того чтобы загружать всю последовательность в память

Важно



Все **генераторы** являются **итераторами**, но не все **итераторы** являются **генераторами**

Экспресс-опрос

?

Что используется для создания генераторов?

Что используется для создания генераторов

1

Генераторные функции

С использованием ключевого слова `yield`

2

Генераторные выражения

Создание генераторов в круглых скобках
(...)

Основные особенности генераторов



Ленивые вычисления



Не занимают много памяти



Сохраняют состояние



Используют `yield` вместо `return`

Использование генератора



Генераторы удобны для **обработки больших файлов, создания бесконечных последовательностей и работы с потоками данных**



Функция с `yield`

Это функция, которая возвращает объект генератора

Экспресс-опрос

?

Каковы особенности функции с yield?

Особенности функции с `yield`



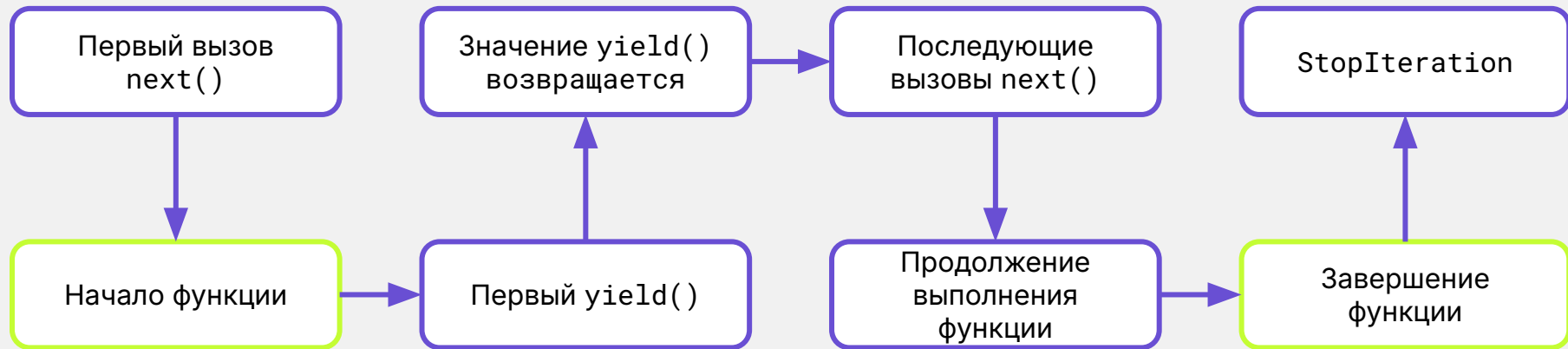
При вызове `next()`

Выполнение функции **приостанавливается на `yield`**

При следующем вызове

Выполнение функции **продолжается с того же места**

Как работает yield



Как работает yield



Синтаксис

```
def generator():  
    yield value
```

Пояснение

- `def generator():` – объявление функции, которая вернет генератор
- `yield` – ключевое слово, приостанавливающее выполнение функции и возвращающее значение
- `value` – результат, который будет передан при вызове `next()`

Пример генератора с yield

```
def generate_values():
    print("Начало работы")
    yield 1 # Приостанавливаем выполнение и возвращаем 1
    print("Продолжение работы")
    yield 2 # Приостанавливаем выполнение и возвращаем 2
    print("Завершение работы")
```

```
gen = generate_values() # Создаём генератор, но код внутри функции пока
не выполняется
```

```
print(next(gen)) # Начало работы → 1
print(next(gen)) # Продолжение работы → 2
print(next(gen)) # Завершение работы → StopIteration, так как нет
третьего yield
```

Экспресс-опрос

?

Что такое функция-генератор?



Функция-генератор

Такая функция может принимать аргументы, которые передаются однократно при создании генератора и используются для генерации значений

Функция-генератор



Синтаксис

```
def generator(param1, param2):  
    yield value
```

Пояснение

param1, param2 – аргументы, передаваемые при создании генератора

Пример: генератор чисел "n" раз

```
def count_up_to(n):
    count = 1
    while count <= n:
        yield count # Возвращаем текущее значение и "замораживаем"
# выполнение
        count += 1 # После следующего вызова next() продолжится отсюда

gen = count_up_to(5)

print(next(gen))
print(next(gen))
print(next(gen))
print(next(gen))
print(next(gen))
# print(next(gen)) # StopIteration
```

Важно



Генератор можно **перебирать в цикле for**, как и любой итератор. В этом случае **for автоматически вызывает next()**, пока генератор **не исчерпает все значения**

Пример: генератор и for

```
def count_up_to(n):
    count = 1
    while count <= n:
        yield count
        count += 1
```

```
gen = count_up_to(5)
```

```
for number in gen:
    print(number)
```

Экспресс-опрос

?

Что такое генераторное выражение?



Генераторное выражение

Это способ создания последовательности значений без предварительного вычисления всех элементов. Вместо хранения данных в памяти оно вычисляет каждый следующий элемент только при необходимости

Особенности генераторного выражения



Генераторное выражение возвращает объект генератора, который является итератором и поддерживает метод `__next__()`, позволяя получать элементы по одному.

Оно похоже на списковое включение, но использует круглые скобки (`...`) вместо квадратных [`...`]

Характеристика	Списковое включение [...]	Генераторное выражение (...)
Создаваемый объект	Список (<code>list</code>)	Генератор (<code>generator</code>)
Создание элементов	Все элементы вычисляются сразу и хранятся в памяти	Каждый элемент создаётся только в момент запроса
Использование памяти	Требует хранения всей последовательности	Экономит память, так как не хранит данные
Доступ к элементам	Можно обращаться по индексу (<code>list[0]</code>)	Доступ возможен только через <code>next()</code> или в <code>for</code>
Подходит для	Малых и средних последовательностей	Больших последовательностей и потоковой обработки данных

Синтаксис генераторного выражения

```
generator = (expression for item in iterable)
```

Пример 1: Создание генераторного выражения

```
# Генерирует квадраты чисел от 0 до 4
squares = (x ** 2 for x in range(5))
```

```
print(squares) # Объект генератора
```

```
print(next(squares))
print(next(squares))
print(next(squares))
print(next(squares))
print(next(squares))
```

Экспресс-опрос

?

Каковы особенности генераторного выражения?

Особенности создания генераторного выражения



- Возвращает генератор, а не сразу список значений
- `next()` вычисляет следующий элемент только при запросе

Пример 2: Генераторное выражение в for

```
squares = (x ** 2 for x in range(5))
```

```
for num in squares:  
    print(num)
```

Особенности генераторного выражения в for



- Генератор автоматически перебирается в for, без явного вызова `next()`
- После прохода по всем элементам он исчерпывается и повторно использовать его нельзя

Пример 3: Потребление памяти

```
import sys

list_comp = [x**2 for x in range(10**6)] # Списковое включение
gen_expr = (x**2 for x in range(10**6)) # Генераторное выражение

print("Размер списка:", sys.getsizeof(list_comp), "байт")
print("Размер генератора:", sys.getsizeof(gen_expr), "байт")
```

Особенности потребления памяти генераторным выражением



- Список занимает много памяти, так как хранит все элементы сразу
- Генератор почти не занимает памяти, так как создаёт элементы по мере запроса

Использование генераторных выражений



Генераторные выражения можно передавать в аргументы функций, где ожидается итерируемый объект. Это особенно полезно при обработке больших последовательностей

Пример: Генераторное выражение в any() и all()

```
words = ["apple", "Banana", "cherry", "Apricot"]  
  
print(any(word[0].isupper() for word in words)) # Есть слово с заглавной  
буквы  
print(all(len(word) > 3 for word in words)) # Все слова длиннее 3 букв
```



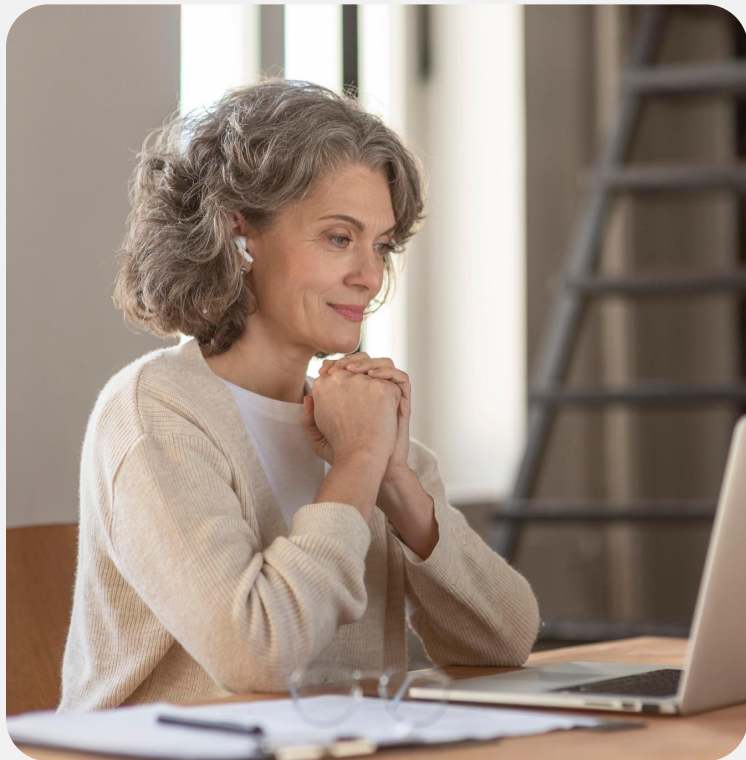
ВОПРОСЫ





ЗАДАНИЕ





Выберите верный вариант ответа

1.1 Какой разделитель используется по умолчанию в CSV-файлах?

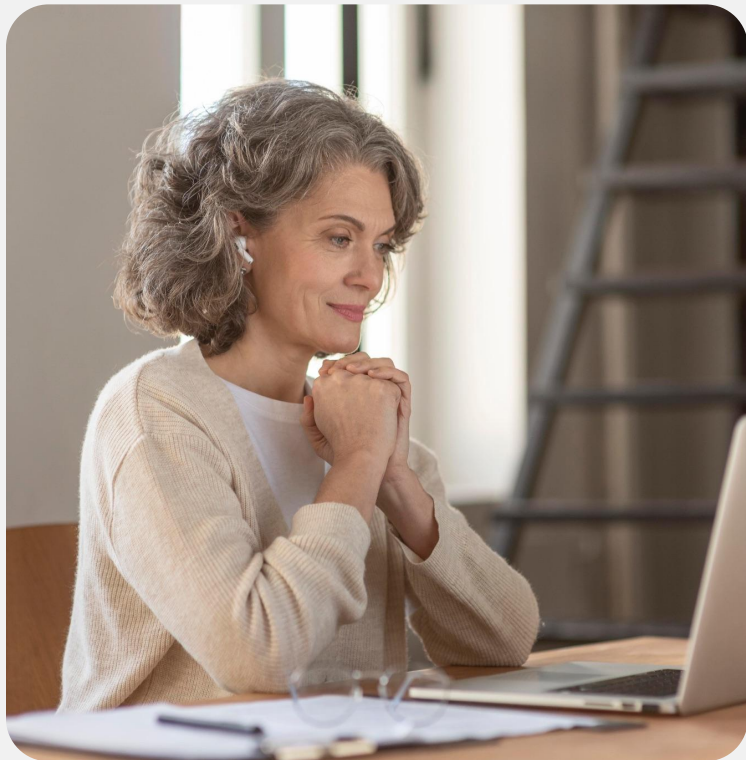
- a. ;
- b. \t
- c. ,
- d. :



Выберите верный вариант ответа

1.1 Какой разделитель используется по умолчанию в CSV-файлах?

- a. ;
- b. \t
- c. ,
- d. :



Выберите верный вариант ответа

1.2 Какой метод используется для чтения CSV-файла в словарь?

- a. `csv.reader()`
- b. `csv.DictReader()`
- c. `csv.writer()`
- d. `csv.DictWriter()`



Выберите верный вариант ответа

1.2 Какой метод используется для чтения CSV-файла в словарь?

- a. `csv.reader()`
- b. `csv.DictReader()`
- c. `csv.writer()`
- d. `csv.DictWriter()`

Выберите верный вариант ответа

1.3 Какой результат будет получен при выполнении следующего кода?

```
import csv
data = [['name', 'age'], ['Alice', 25], ['Bob', 30]]
with open('test.csv', 'w', newline='') as f:
    writer = csv.writer(f, delimiter=';')
    writer.writerows(data)

with open('test.csv') as f:
    reader = csv.reader(f, delimiter=';')
    for row in reader:
        print(row)
```

- a. ['name', 'age']
['Alice', 25]
['Bob', 30]
- b. ['name;age']
['Alice;25']
['Bob;30']
- c. ['name', 'age']
['Alice', '25']
['Bob', '30']
- d. ['Alice', '25']
['Bob', '30']

Выберите верный вариант ответа

1.3 Какой результат будет получен при выполнении следующего кода?

```
import csv
data = [['name', 'age'], ['Alice', 25], ['Bob', 30]]
with open('test.csv', 'w', newline='') as f:
    writer = csv.writer(f, delimiter=';')
    writer.writerows(data)

with open('test.csv') as f:
    reader = csv.reader(f, delimiter=';')
    for row in reader:
        print(row)
```

- a. ['name', 'age']
['Alice', 25]
['Bob', 30]
- b. ['name;age']
['Alice;25']
['Bob;30']
- c. ['name', 'age']
['Alice', '25']
['Bob', '30']
- d. ['Alice', '25']
['Bob', '30']



Выберите верный вариант ответа

1.4 Как можно заменить пропущенные значения на None при чтении CSV-файла?

- a. Использовать параметр `skip_blank_lines=True`
- b. Использовать параметр `none_for_empty=True`
- c. Вручную пройти по строкам и заменить пустые строки на None
- d. Использовать параметр `fill_empty=None`



Выберите верный вариант ответа

1.4 Как можно заменить пропущенные значения на None при чтении CSV-файла?

- a. Использовать параметр `skip_blank_lines=True`
- b. Использовать параметр `none_for_empty=True`
- c. Вручную пройтись по строкам и заменить пустые строки на None
- d. Использовать параметр `fill_empty=None`

Выберите верный вариант ответа

1.5 Какой результат будет получен при выполнении следующего кода?

```
import csv
with open('data.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['name', 'age'])
    writer.writerow(['Alice', '25'])
    writer.writerow(['Bob', '30'])

with open('data.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row['name'], row['age'])
```

- a. Alice 25
Bob 30
- b. Alice '25'
Bob '30'
- c. Alice Bob
'25' '30'
- d. KeyError

Выберите верный вариант ответа

1.5 Какой результат будет получен при выполнении следующего кода?

```
import csv
with open('data.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['name', 'age'])
    writer.writerow(['Alice', '25'])
    writer.writerow(['Bob', '30'])

with open('data.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row['name'], row['age'])
```

- a. Alice 25
Bob 30
- b. Alice '25'
Bob '30'
- c. Alice Bob
'25' '30'
- d. KeyError



Выберите верный вариант ответа

1.6 Какой метод модуля statistics используется для вычисления среднего арифметического?

- a. `mean()`
- b. `median()`
- c. `mode()`
- d. `harmonic_mean()`



Выберите верный вариант ответа

1.6 Какой метод модуля statistics используется для вычисления среднего арифметического?

- a. `mean()`
- b. `median()`
- c. `mode()`
- d. `harmonic_mean()`



Выберите верный вариант ответа

1.7 Что будет напечатано в результате выполнения следующего кода?

```
import statistics
data = [1, 2, 2, 3, 4]
print(statistics.mode(data))
```

- a. 1
- b. 2
- c. 3
- d. Ошибка, так как мода не определена



Выберите верный вариант ответа

1.7 Что будет напечатано в результате выполнения следующего кода?

```
import statistics
data = [1, 2, 2, 3, 4]
print(statistics.mode(data))
```

- a. 1
- b. 2
- c. 3
- d. Ошибка, так как мода не определена



Выберите верный вариант ответа

1.8 Какой результат будет получен при выполнении следующего кода?

```
import statistics
data = [10, 20, 30, 40, 50]
print(statistics.quantiles(data, n=4))
```

- a. [10, 20, 30, 40, 50]
- b. [10, 25, 35, 50]
- c. [10, 20, 30, 40]
- d. [15, 25, 35, 45]



Выберите верный вариант ответа

1.8 Какой результат будет получен при выполнении следующего кода?

```
import statistics
data = [10, 20, 30, 40, 50]
print(statistics.quantiles(data, n=4))
```

- a. [10, 20, 30, 40, 50]
- b. [10, 25, 35, 50]
- c. [10, 20, 30, 40]
- d. [15, 25, 35, 45]



ВОПРОСЫ





Выберите верные варианты ответа

2.1 Какой код создаёт итератор?

- a. `numbers = iter([1, 2, 3])`
- b. `numbers = [1, 2, 3].iter()`
- c. `numbers = next([1, 2, 3])`
- d. `numbers = [1, 2, 3].__iter__()`



Выберите верные варианты ответа

2.1 Какой код создаёт итератор?

- a. `numbers = iter([1, 2, 3])`
- b. `numbers = [1, 2, 3].iter()`
- c. `numbers = next([1, 2, 3])`
- d. `numbers = [1, 2, 3].__iter__()`



Выберите верный вариант ответа

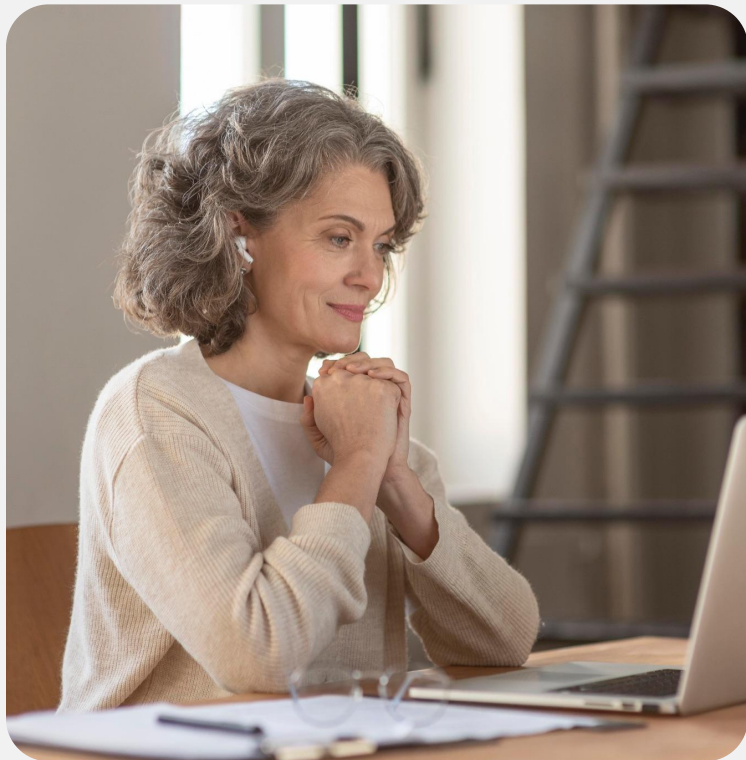
2.2 Какие числа выведет следующий код?

```
numbers = [2, 4, 6, 8]
iterator = iter(numbers)
```

```
print(next(iterator))
print(next(iterator))
```

```
for num in iterator:
    print(num)
```

- a. 2, 4
- b. 2, 4, 6, 8
- c. 2, 4, 6, 8, 6, 8
- d. 2, 4, 6, 8, 2, 4, 6, 8



Выберите верный вариант ответа

2.2 Какие числа выведет следующий код?

```
numbers = [2, 4, 6, 8]
iterator = iter(numbers)
```

```
print(next(iterator))
print(next(iterator))
```

```
for num in iterator:
    print(num)
```

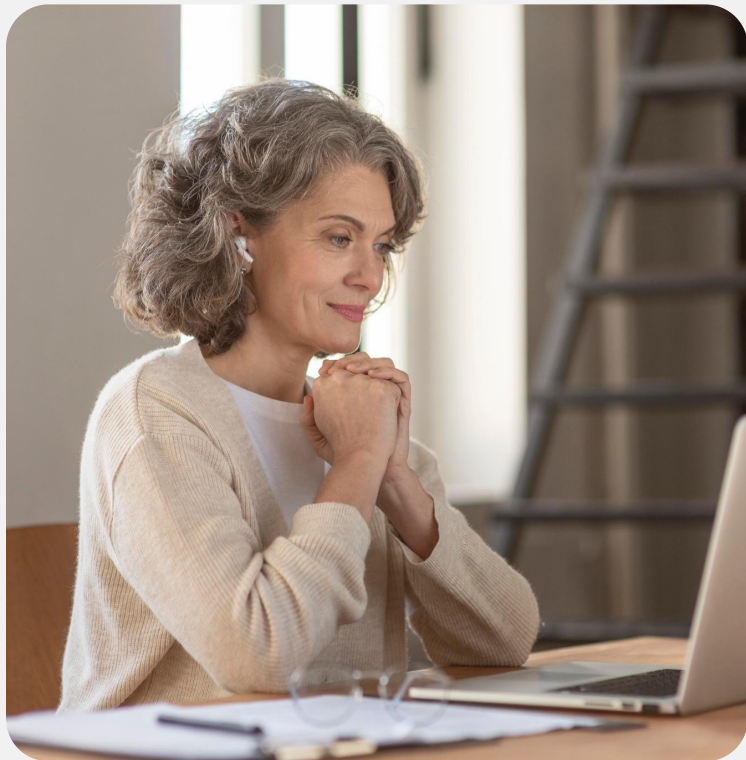
- a. 2, 4
- b. 2, 4, 6, 8**
- c. 2, 4, 6, 8, 6, 8
- d. 2, 4, 6, 8, 2, 4, 6, 8



Выберите верные варианты ответа

2.3 Какие из этих объектов являются итерируемыми?

- a. `list`
- b. `dict`
- c. `set`
- d. `int`
- e. `str`



Выберите верные варианты ответа

2.3 Какие из этих объектов являются итерируемыми?

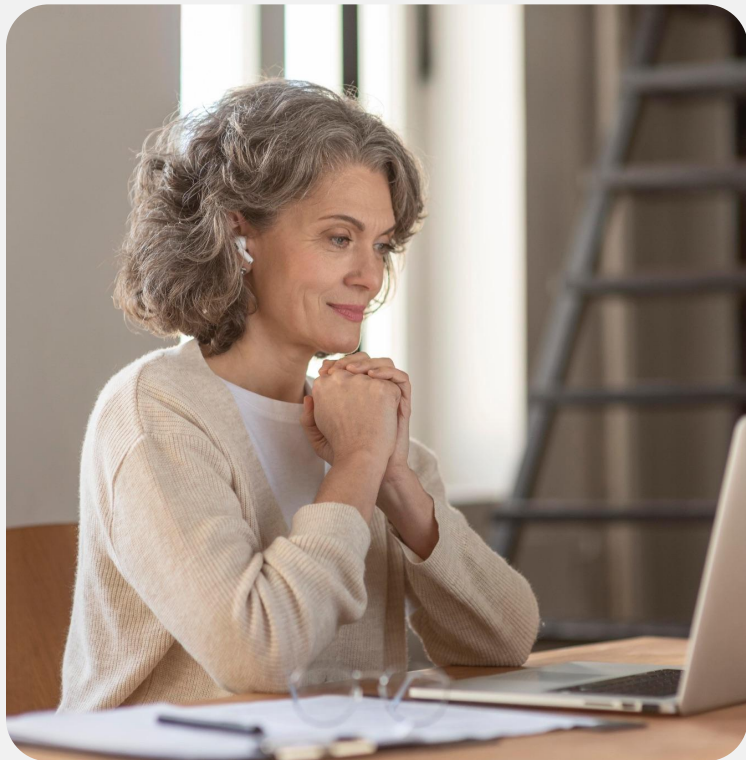
- a. list
- b. dict
- c. set
- d. int
- e. str



Выберите верные варианты ответа

2.4 Какие утверждения о итераторах верны?

- a. Итератор можно создать из любого объекта
- b. `next(iterable)` работает без `iter()`
- c. Итератор можно создать из списка с помощью `iter()`
- d. Все итерируемые объекты являются итераторами
- e. Итератор хранит все элементы в памяти
- f. Итератор можно перебирать только один раз
- g. Цикл `for` внутри себя использует `iter()` и `next()`
- h. Итератор можно создать с помощью `iter(iterable)`



Выберите верные варианты ответа

2.4 Какие утверждения о итераторах верны?

- a. Итератор можно создать из любого объекта
- b. `next(iterable)` работает без `iter()`
- c. Итератор можно создать из списка с помощью `iter()`
- d. Все итерируемые объекты являются итераторами
- e. Итератор хранит все элементы в памяти
- f. Итератор можно перебирать только один раз
- g. Цикл `for` внутри себя использует `iter()` и `next()`
- h. Итератор можно создать с помощью `iter(iterable)`



Выберите верные варианты ответа

2.5 Какие функции из `itertools` создают бесконечные итераторы?

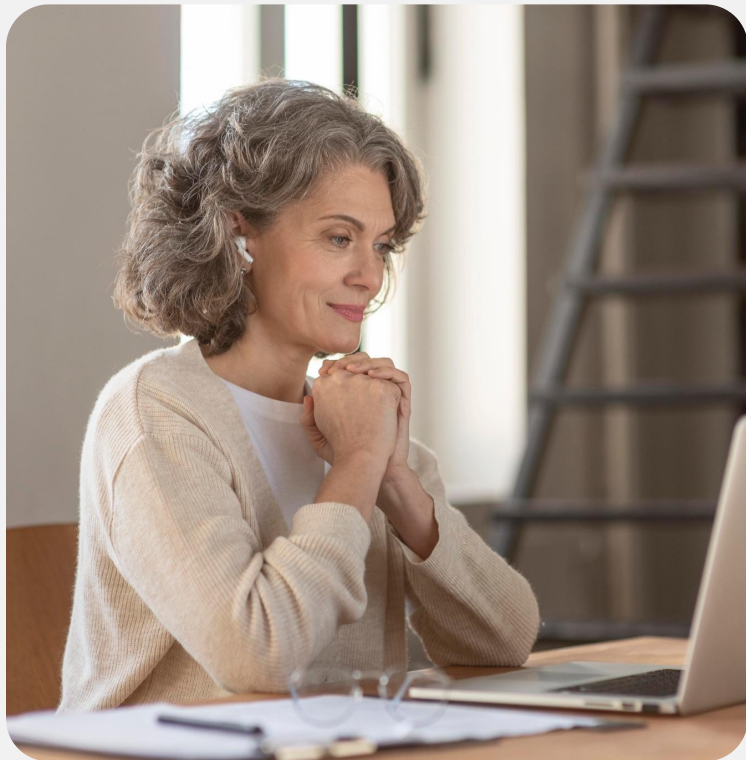
- a. `count()`
- b. `cycle()`
- c. `permutations()`
- d. `product()`
- e. `chain()`



Выберите верные варианты ответа

2.5 Какие функции из `itertools` создают бесконечные итераторы?

- a. `count()`
- b. `cycle()`
- c. `permutations()`
- d. `product()`
- e. `chain()`



Выберите верный вариант ответа

2.6 Что напечатает следующий код?

```
import itertools
```

```
pairs = itertools.product([1, 2], ["A", "B"])  
print(list(pairs))
```

- `[(1, 2), ('A', 'B')]`
- `[(1, 'A'), (2, 'B')]`
- `[(1, 'A'), (1, 'B'), (2, 'A'), (2, 'B')]`
- Ошибка, так как `product()` требует числовые данные



Выберите верный вариант ответа

2.6 Что напечатает следующий код?

```
import itertools
```

```
pairs = itertools.product([1, 2], ["A", "B"])  
print(list(pairs))
```

- a. [(1, 2), ('A', 'B')]
- b. [(1, 'A'), (2, 'B')]
- c. [(1, 'A'), (1, 'B'), (2, 'A'), (2, 'B')]
- d. Ошибка, так как product() требует числовые данные



Выберите верные варианты ответа

2.7 Чем генератор отличается от обычной функции?

- a. Генератор использует `yield`, а не `return`
- b. Генератор возвращает сразу все значения
- c. Генератор запоминает состояние между вызовами
- d. Генератор создаёт бесконечные последовательности



Выберите верные варианты ответа

2.7 Чем генератор отличается от обычной функции?

- a. Генератор использует `yield`, а не `return`
- b. Генератор возвращает сразу все значения
- c. Генератор запоминает состояние между вызовами
- d. Генератор создаёт бесконечные последовательности



Выберите верный вариант ответа

2.8 Что выведет следующий код?

```
def squares(n):
    for i in range(n):
        yield i ** 2
```

```
gen = squares(4)
```

```
for num in gen:
    print(num)
```

- a. 0, 1, 4, 9
- b. 0, 1, 4, 9, StopIteration



Выберите верный вариант ответа

2.8 Что выведет следующий код?

```
def squares(n):
    for i in range(n):
        yield i ** 2
```

```
gen = squares(4)
```

```
for num in gen:
    print(num)
```

- a. 0, 1, 4, 9
- b. 0, 1, 4, 9, StopIteration



Выберите верные варианты ответа

2.9 Какие из следующих утверждений о генераторах верны?

- a. Генератор вычисляет элементы по запросу
- b. Генератор можно вызвать несколько раз и он всегда начинает сначала
- c. Генератор можно использовать в `for`
- d. Генератор занимает меньше памяти, чем список



Выберите верные варианты ответа

2.9 Какие из следующих утверждений о генераторах верны?

- a. Генератор вычисляет элементы по запросу
- b. Генератор можно вызвать несколько раз и он всегда начинает сначала
- c. Генератор можно использовать в `for`
- d. Генератор занимает меньше памяти, чем список



Выберите верный вариант ответа

2.10 Чем генераторное выражение отличается от спискового включения?

- a. Генератор загружает все элементы в память
- b. Генератор вычисляет элементы только при запросе
- c. Генераторное выражение использует квадратные скобки
- d. Генератор поддерживает доступ к элементам по индексу



Выберите верный вариант ответа

2.10 Чем генераторное выражение отличается от спискового включения?

- a. Генератор загружает все элементы в память
- b. Генератор вычисляет элементы только при запросе
- c. Генераторное выражение использует квадратные скобки
- d. Генератор поддерживает доступ к элементам по индексу

Выберите верные варианты ответа

2.11 Какой код создаст генератор для квадратов чисел от 1 до 5?

- a.

```
def squares():  
    yield [x**2 for x in range(1, 6)]
```
- b.

```
def squares():  
    for x in range(1, 6):  
        yield x**2
```
- c.

```
squares = [x**2 for x in range(1, 6)]
```
- d.

```
squares = (x**2 for x in range(1, 6))
```

Выберите верные варианты ответа

2.11 Какой код создаст генератор для квадратов чисел от 1 до 5?

a. `def squares():
 yield [x**2 for x in range(1, 6)]`

b. `def squares():
 for x in range(1, 6):
 yield x**2`

c. `squares = [x**2 for x in range(1, 6)]`

d. `squares = (x**2 for x in range(1, 6))`

Выберите верные варианты ответа

2.12 Какой код создаст бесконечный генератор?

- a.

```
def infinite_loop():
    while True:
        yield 1
```
- b.

```
def infinite_loop():
    yield from range(10)
```
- c.

```
def infinite_loop():
    for i in range(10):
        yield i
```
- d.

```
import itertools

def infinite_loop():
    yield from itertools.cycle([1, 2, 3])
```
- e.

```
def infinite_loop():
    while True:
        yield from range(10)
```

Выберите верные варианты ответа

2.12 Какой код создаст бесконечный генератор?

a. `def infinite_loop():`

`while True:`

`yield 1`

b. `def infinite_loop():`

`yield from range(10)`

c. `def infinite_loop():`

`for i in range(10):`

`yield i`

d. `import itertools`

`def infinite_loop():`

`yield from itertools.cycle([1, 2, 3])`

e. `def infinite_loop():`

`while True:`

`yield from range(10)`



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА



1. Поиск чисел, кратных 7

Используйте **генераторное выражение**, чтобы найти все числа кратные **7** в диапазоне от **1** до **100**.

Пример вывода:

7 14 21 28 35 42 49 56 63 70 77 84 91 98

2. Фильтрация слов по длине

Используйте **генераторное выражение**, чтобы найти все слова, длина которых больше 5.

Исходные данные:

```
words = ["table", "computer", "pen", "keyboard", "mouse", "phone"]
```

Пример вывода:

```
computer keyboard mouse
```



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

Работа с CSV-файлами

На лекции при выполнении практического задания 2 был получен файл `grades.csv`.

Напишите программу, которая:

1. Читает этот файл.
2. Создает на его основе три новых файла: `grades-Science.csv`, `grades-Math.csv`, `grades-Physics.csv`. В каждом из этих файлов только два столбца - имя и количество баллов по тому предмету, который указан в названии файла.
3. Создает четвертый файл `grades-info.csv`. В этом файле три строки (по названиям предметов) и столбцы со статистическими характеристиками оценок: среднее арифметическое, минимум, максимум, медиана, стандартное отклонение.



ВОПРОСЫ



Домашнее задание

1. Комбинации одежды

Напишите функцию, которая принимает списки типов одежды, цветов и размеров, а затем **генерирует все возможные комбинации** в формате "Clothe - Color - Size".

Данные:

```
clothes = ["T-shirt", "Jeans", "Jacket"]
colors = ["Red", "Blue", "Black"]
sizes = ["S", "M", "L"]
```

Пример вывода:

```
T-shirt - Red - S
T-shirt - Red - M
T-shirt - Red - L
T-shirt - Blue - S
...
Jacket - Black - L
```

Домашнее задание

2. Генератор уникальных элементов

Создайте **генератор**, который принимает **список элементов** и **выдаёт только уникальные значения**, сохраняя **порядок их появления** в исходном списке.

Данные:

```
data = [3, 1, 2, 3, 4, 1, 5, 2, 6, 7, 5, 8]
```

Пример вывода:

```
3
1
2
4
5
6
7
8
```



ВОПРОСЫ



Заключение

